

Python objects

Typen af værdier

"Banana" # str

117 # int

3.1415 # float

True # bool

Ny værdi af en anden type

```
str(42)           # returns "42"  
str(3.1415)       # returns "3.1415"  
  
int(3.1415)       # returns 3  
int(True)         # returns 1  
  
int("42")         # returns 42  
float("3.1415")   # returns 3.1415
```

Python værdier er objekter

Værdier er “objekter”

"banana"

Metoder

"banana".upper()

Metoder

"banana".upper()

↓ substitution

"BANANA"

Funktioner vs. metoder

Hey len function, work on this string!

```
print(len("Banana"))
```

Hey string, use your upper method!

```
print("Banana".upper())
```

Metoder

"Hej {}"

Metoder

```
"Hej {} ".format("Ib")
```

Metoder

```
"Hej {}" .format( "Ib" )
```

↓

substitution

```
"Hej Ib"
```

format

```
p = "Hej {}".format("Mogens")  
print(p)
```

format

```
navn = "Mogens"  
p = "Hej {}".format(navn)  
print(p)
```

format

```
navn = "Mogens"  
s = "Hej {}"  
p = s.format(navn)  
print(p)
```

Tanketjek

```
navn = "Mogens"  
s = "Hej {}"  
p = s.format(s).format(navn)  
print(p)
```

Tanketjek

```
navn = "Mogens"  
s = "Hej {}"  
p = s.format(s).format(navn)
```

Tanketjek

```
navn = "Mogens"
```

```
s = "Hej {}"
```

```
p = s.format(s).format(navn)
```

substitution



```
"Hej {}".format("Hej {}".format("Mogens"))
```

Tanketjek

```
navn = "Mogens"
```

```
s = "Hej {}"
```

```
p = s.format(s).format(navn)
```

substitution

```
"Hej {} ".format("Hej {} ").format("Mogens")
```

```
"Hej Hej {} ".format("Mogens")
```

Tanketjek

```
navn = "Mogens"
```

```
s = "Hej {}"
```

```
p = s.format(s).format(navn)
```

substitution

```
"Hej {}".format("Hej {}").format("Mogens")
```

```
"Hej Hej {}".format("Mogens")
```

```
"Hej Hej Mogens"
```

format

```
request = "{} kugler {} tak"  
n = 2  
kind = "vanilje"  
s = request.format(n, kind)  
print(s)
```

format

```
request = "Hit med {} kugler {}!"  
n = 2  
kind = "vanilje"  
s = request.format(n, kind)  
print(s)
```

String formatting

```
# control decimals
```

```
"{: .4f}".format(1/3)
```

```
# right-justify with spaces:
```

```
"{: >10}".format('Banana')
```

```
# both at the same time:
```

```
"{: >20.2f} DKK".format(1000.1234567)
```

in og not in

```
dna = "ATTCAGCGAGAGGAGGGTACAGTCAC"  
motif = "AGGAGG"
```

```
print(motif in dna)
```

```
# under the hood:
```

```
print(dna.__contains__(motif))
```

Get characters from a string

```
fruit = "Orange"
```

```
letter = fruit[1]
```

```
print(letter)
```

Get characters from a string

```
fruit = "Orange"
```

```
letter = fruit[1]
```



```
"Orange"[1]
```

substitution

```
print(letter)
```

Get characters from a string

```
fruit = "Orange"
```

```
letter = fruit[1]
```



```
"Orange"[1]
```

substitution



```
"r"
```

substitution

```
print(letter)
```

Get characters from a string

```
fruit = "Orange"
```

```
fruit[0] # O
```

```
fruit[1] # r
```

```
fruit[2] # a
```

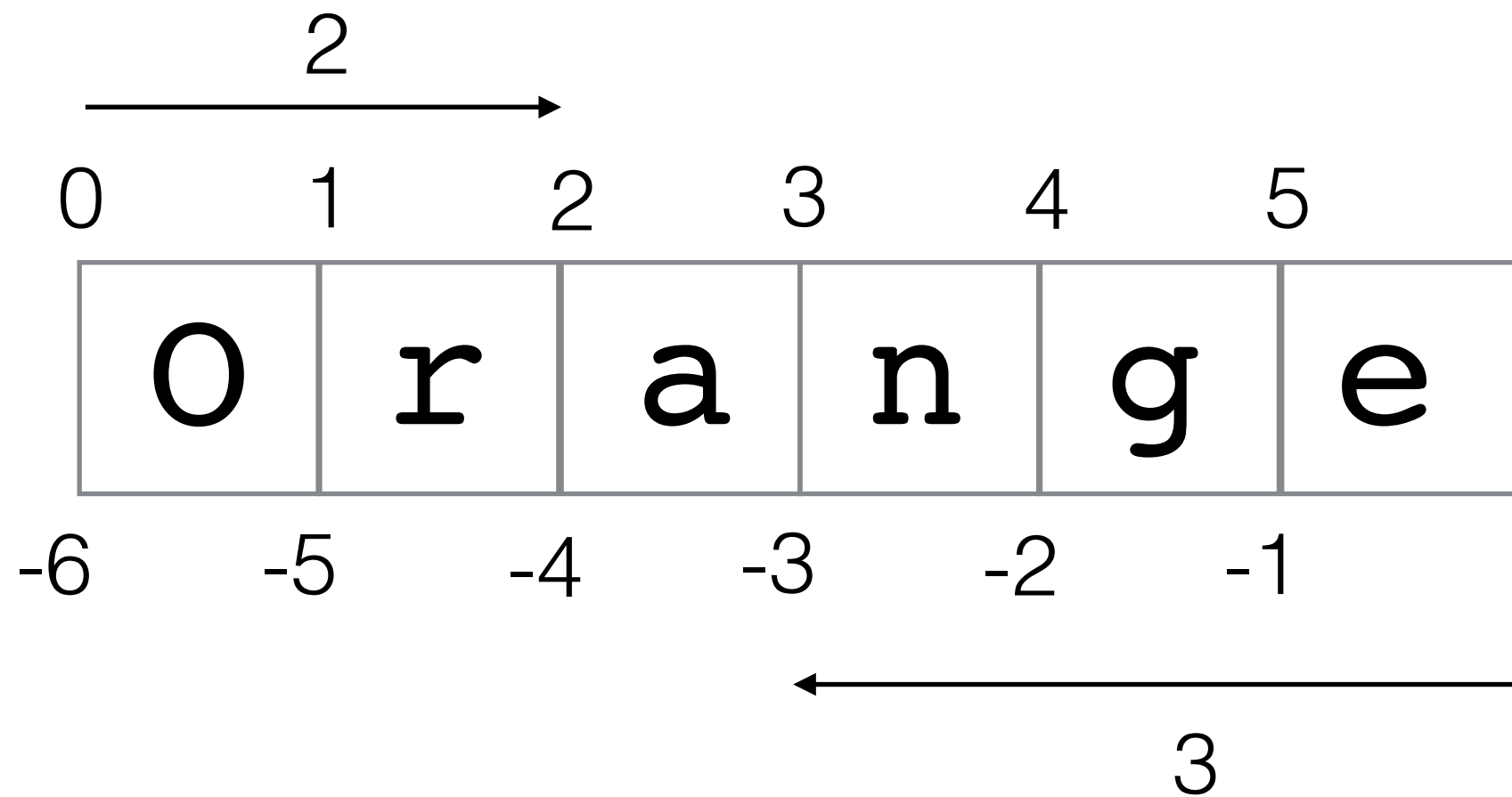
```
fruit[3] # n
```

```
fruit[4] # g
```

```
fruit[5] # e
```

```
fruit[len(fruit)-1] # e
```

Indexing



Negative indexes

```
fruit = "Orange"
```

```
fruit[-6] # O
```

```
fruit[-5] # r
```

```
fruit[-4] # a
```

```
fruit[-3] # n
```

```
fruit[-2] # g
```

```
fruit[-1] # e
```

IndexError

```
fruit = "Orange"
```

```
print(fruit[6])
```

Slicing

- Et interval gives som startindex og slutindex med kolon imellem.
- Slutindex er ikke inkluderet i intervaller.

"GGGTTTCCC" [3 : 6]

Slicing

- Et interval gives som startindex og slutindex med kolon imellem.
- Slutindex er ikke inkluderet i intervaller.

"GGGTTTCCC" [3 : 6]

↓ substitution

"TTT"

Slicing

```
dna = 'GGGTTTCCC'  
codon = dna[3:6]
```

```
print(codon)
```

Slicing

```
dna = 'GGGTTTCCC'  
codon = dna[3:6]  
      └───┬─── substitution  
          ↓  
      "TTT"  
  
print(codon)
```

Slicing

```
pie = 'Piece1Piece2Piece3Piece4'
```

0 6 12 18

```
print(pie[0:6])
```

```
print(pie[6:12])
```

```
print(pie[12:18])
```

```
print(pie[18:24])
```

Slicing

```
pie = 'Piece1Piece2Piece3Piece4'

print(pie[12:])
print(pie[:12])
print(pie[:])
print(pie[::6])
print(pie[::-1])
```

Slicing

```
pie[start:end]
```

```
pie[0:len(pie)] # default
```

```
pie[start:end:step]
```

```
pie[0:len(pie):1] # default
```

Beware of slicing thin air

```
pie = 'Orange'
```

```
print(pie[2:999])
```

```
print(pie[9:12])
```

Streng er “immutable”

- Man kan ikke ændre streng:

```
fruit = 'banana'  
fruit[0] = 'B'    # not legal !!
```

- Man på lave en ny streng:

```
new_fruit = 'B' + fruit[1:]  
# eller bedre:  
new_fruit = fruit.capitalize()
```

Tupler

(2 , 5 , 3 , 9)

(' Blue ' , ' Red ')

(42 , 3.1415 , ' Bob ')

Lists

Lister

list

Lister af værdier

[2 , 5 , 3 , 9]

["C" , "B" , "C"]

Hvad er en listeværdi?



Definition af en liste

`l = []`

`l = [5.1, 3.2, 7.4]`

`l = ["red", "blue", "green"]`

`l = [[5, 6], [2, 4]]`

Length of a list

```
len([3, 7, 4])
```

Under the hood

```
len([3, 7, 4])
```

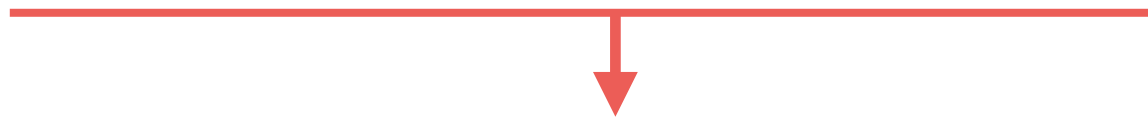
```
# [3, 7, 4].__len__()
```

Adding lists

`[ 3 , 5 ] + [ 4 , 2 ]`

Adding lists

$[3, 5] + [4, 2]$



$[3, 5, 4, 2]$

reduktion

Under the hood

`[3, 5] + [4, 2]`

`# [3, 5].__add__([4, 2])`

List membership

```
colors = ["red", "blue", "green"]
```

```
if "blue" in colors:  
    print("great!")
```

```
if "yellow" not in colors:  
    print("don't like it anyway")
```

Under the hood

```
5 in [3, 5, 7]
```

```
# [3, 5, 7].__contains__(5)
```



append

```
numbers = [1, 2, 3]
```

```
val = 7
```

```
numbers.append(val)
```

```
print(numbers)
```

append

```
numbers = [1, 2, 3]
```

```
val = 7
```

```
numbers.append(val)
```

↓ ↓

```
[1, 2, 3].append(7)
```

substitution

```
print(numbers)
```

append

```
numbers = [1, 2, 3]
```

```
val = 7
```

```
numbers.append(val)
```

↓ ↓

```
[1, 2, 3].append(7)
```

↓

```
None
```

substitution

substitution

```
print(numbers)
```

append

```
numbers = [1, 2, 3]
```

```
val = 7
```

```
numbers.append(val)
```

↓ ↓

```
[1, 2, 3].append(7)
```

None

substitution

substitution

```
print(numbers)
```

Hvad har append gjort ved numbers?

extend

```
numbers = [1, 2, 3]  
more_numbers = [5, 11]  
numbers.extend(more_numbers)  
  
print(numbers)
```

extend

```
numbers = [1, 2, 3]
```

```
more_numbers = [5, 11]
```

```
numbers.extend(more_numbers)
```

```
[1, 2, 3].extend([5, 11])
```

substitution

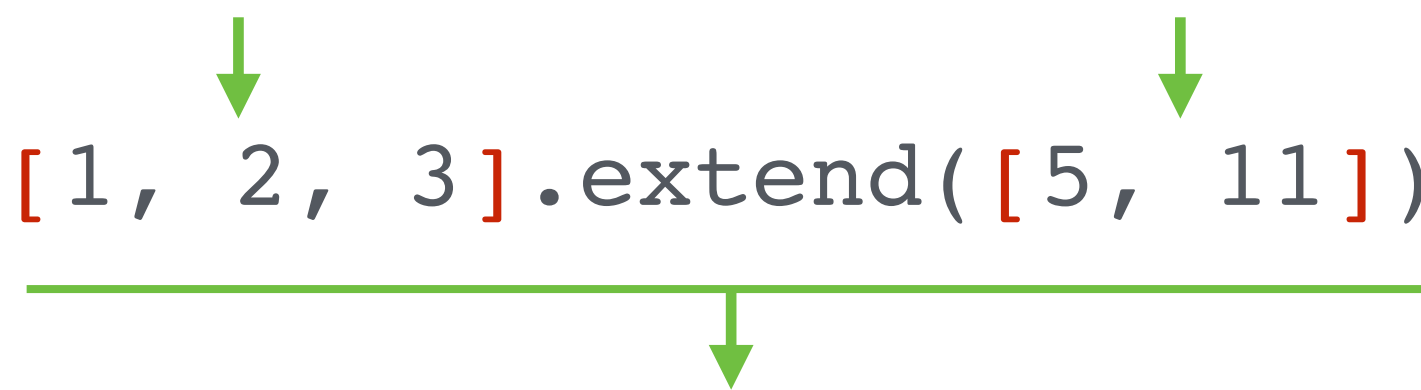
```
print(numbers)
```

extend

```
numbers = [1, 2, 3]
```

```
more_numbers = [5, 11]
```

```
numbers.extend(more_numbers)
```



```
[1, 2, 3].extend([5, 11])
```

None

substitution

substitution

```
print(numbers)
```

extend

```
numbers = [1, 2, 3]
```

```
more_numbers = [5, 11]
```

```
numbers.extend(more_numbers)
```



```
[1, 2, 3].extend([5, 11])
```

None

substitution

substitution

```
print(numbers)
```

Hvad har `extend` gjort ved `numbers`?

pop

```
numbers = [1, 2, 3]  
val = numbers.pop(1)
```

```
print(val)  
print(numbers)
```

pop

```
numbers = [1, 2, 3]  
val = numbers.pop(1)
```



2

substitution

```
print(val)  
print(numbers)
```

pop

```
numbers = [1, 2, 3]  
val = numbers.pop(1)
```



2

substitution

```
print(val)  
print(numbers)
```

Hvad har `pop` gjort ved `numbers`?

Indexing

0	1	2	3	4
['A' ,	'B' ,	'C' ,	'D' ,	'E']
-5	-4 ,	-3 ,	-2 ,	-1

Accessing entries in a list

```
colors = ["red", "blue", "white"]
```

```
colors[0] # red
```

```
colors[1] # blue
```

```
colors[2] # white
```

```
colors[-1] # white
```

0	1	2	3	4
['A',	'B',	'C',	'D',	'E']
-5	-4,	-3,	-2,	-1

Accessing entries in a list

```
["red", "blue", "white"][0] # red  
["red", "blue", "white"][1] # blue  
["red", "blue", "white"][2] # white
```

```
["red", "blue", "white"][-1] # white
```

0	1	2	3	4
['A',	'B',	'C',	'D',	'E']
-5	-4,	-3,	-2,	-1

Under the hood

```
[3, 5, 7][5]
```

```
# [3, 5, 7].__getitem__(5)
```

Changing entries in a list

```
colors = ["red", "blue", "white"]
```

```
colors[0] = "pink"
```

```
colors[2] = "orange"
```

0	1	2	3	4
['A',	'B',	'C',	'D',	'E']
-5	-4,	-3,	-2,	-1



Tanketjek

```
x = 'Anna'
```

```
y = 'Lotte'
```

```
names = [x, y]
```

```
x = 'Preben'
```

```
print(names) # what is printed here?
```

```
z = names[1]
```

```
z = 'Henning'
```

```
print(names) # and here?
```

```
names[0] = 'Mogens'
```

```
print(names) # and here?
```



Lister af lister

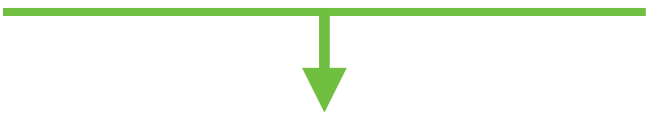
```
matrix = [[11, 12, 13],  
          [21, 22, 23],  
          [31, 32, 33]]
```

```
print(matrix[1])
```

Lister af lister

```
matrix = [[11, 12, 13],  
          [21, 22, 23],  
          [31, 32, 33]]
```

```
print(matrix[1])
```


[21, 22, 23]

substitution



Lister af lister

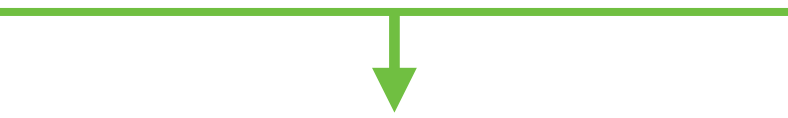
```
matrix = [[11, 12, 13],  
          [21, 22, 23],  
          [31, 32, 33]]
```

```
print(matrix[1][2])
```

Lister af lister

```
matrix = [[11, 12, 13],  
          [21, 22, 23],  
          [31, 32, 33]]
```

```
print(matrix[1][2])
```


[21, 22, 23][2]

substitution

Lister af lister

```
matrix = [[11, 12, 13],  
          [21, 22, 23],  
          [31, 32, 33]]
```

```
print(matrix[1][2])
```

```
[21, 22, 23][2]  
23
```

substitution

substitution

Deleting list elements

```
letters = ["A", "B", "C", "D"]
```

```
del letters[0] # now ["B", "C", "D"]
```

```
del letters[0:2] # now ["D"]
```

Liste til én streng

```
fingers = [ "thumb", "index" ]  
glue = "_"
```

```
# string from list:  
accident = glue.join(fingers)
```

```
# or  
accident = "_".join(fingers)
```





Streng til en liste

```
accident = "thumb_index"
```

```
fingers = accident.split("_")
```

```
print(fingers)
```

Streng til en liste

```
blender_accident = list("thumb_index")
```



Opgave

"ATG"



["A" , "T" , "G"]



"ATG"

Løsning

Løsning

```
# here is a string
```

```
codon = "ATG"
```

```
# turn string into list or characters
```

```
bases = list(codon)
```

```
# use empty string to join elements
```

```
same_codon = "".join(bases)
```

Indexing tuples

`(2, 5, 3, 9)[2]`

`('Blue', 'Red')[1]`

`(42, 3.1514, 'Bob')[ : 2 ]`



Swapping values

Write some code so that **a** gets the value of **b** and **b** gets the value of **a**

a = 1

b = 2

Multiple assignment and unpacking

`a, b = (1, 2)`

`a, b = 1, 2`



Swapping values

Write some code so that **a** gets the value of **b** and **b** gets the value of **a**

a = 1

b = 2

Uge 3

Lists